

Team COiN Lab: A CPU-Only Machine Learning System for Temporal Fraud Detection

Md. Abdur Rahman, Md. Aayat Hossain Mridha

Computational Intelligence Lab, Southeast University, Dhaka, Bangladesh
abdurrahman.etc@gmail.com, ah686634@gmail.com

Abstract—The system submitted by Team COiN Lab to the REACT 2026 Datathon generated fraud probabilities for 262,648 transactions occurring entirely after the labelled period, using raw fields with no historical column provided. The governing decision was to keep the whole system on CPU: the behavioural context a sequence model would have to learn is instead reconstructed explicitly through 250 leakage-safe features computed from strictly earlier rows, while the estimator consists of a rank-averaged blend of two gradient-boosting libraries with one small multilayer perceptron. Twelve threshold rules mined from labelled history are additionally incorporated, and a failure mode hidden by walk-forward validation is quantified: encodings frozen at the training boundary lose 0.0183 average precision across a 62-day forecast, which is mitigated with an encoding-free blend member. The pipeline trains and scores in 73 minutes on four CPU cores, placing 4th of 36 teams at 0.56084 private PR-AUC.

Index Terms—fraud detection, temporal validation, feature engineering, gradient boosting, class imbalance, PR-AUC

I. INTRODUCTION

A transaction is not fraudulent in isolation. A payment of BDT 30,000 is unremarkable for one account and alarming for another. The REACT 2026 Datathon, organised by the IEEE Southeast University Student Branch, was built around that premise: competitors received a chronologically ordered stream of labelled transactions and had to score a later, disjoint window, reconstructing every behavioural signal themselves under explicit rules against target, temporal and test-set leakage.

Two properties of the task constrain any solution. Fraud is rare, at 1.76% of the labelled stream, so the metric is average precision, preferred to ROC-AUC under this imbalance because it responds to the positive class alone [1]. And the scoring window opens the day after the labelled window closes, so shuffled validation lets a customer’s future inform their past and returns an estimate the leaderboard cannot reproduce [2].

A third constraint was self-imposed. On heterogeneous tabular data, tuned tree ensembles remain at least competitive with neural architectures [3]. A deployed scorer is also retrained often and must answer inside an authorisation timeout; one needing a GPU is retrained less often, and against an adapting adversary, retraining cadence is itself an accuracy variable. We fixed a rule before writing code: no stage may require a GPU.

II. METHODOLOGY

A. Dataset Description and Preprocessing

Training holds 731,942 transactions from 2026-01-01 to 2026-07-15 and test holds 262,648 from 2026-07-16 to 2026-09-15, so the two are strictly separated and the forecast horizon reaches 62 days. Each row carries twelve raw fields: four identifiers, a timestamp, an amount in BDT, five categoricals and account age. The label appears in training only, on 1.76% of rows.

Three categorical fields contain intentional gaps. We fill each from the owning entity’s most frequent observed value on labelled history, retaining missingness as its own binary column, since the gaps are not

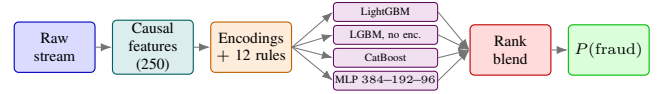


Fig. 1. The proposed CPU-only pipeline. Every operator reads strictly earlier rows; the second member sees no label-derived encoding.

spread evenly across the label. Both files are then concatenated into one frame ordered by timestamp. This is permitted and necessary: an entity’s history must carry across the boundary, so a device first seen in June is still recognised in August. What never crosses is a label. The stream’s first thirty days (107,721 rows) are excluded from every fitting window, since trailing features are still filling there.

B. Causal Feature Engineering

Every feature comes from an operator that can only see strictly earlier rows, so leakage is prevented structurally rather than by inspection. The primitives are group-wise `cumcount` and `cumsum` with the current row subtracted from its own total, group-wise shifts and running extrema, and half-open trailing windows $[t - w, t)$ evaluated by binary search over per-key prefix sums. All 250 columns are built from these.

Amount normality asks whether the amount is unusual for the parties involved: running z -scores of $\log(1 + \text{amount})$ per customer, merchant, category and location, plus the fraction of that customer’s past above each of five global amount quantiles. *Tempo* asks whether the timing is unusual: inter-arrival gaps at lags one, two and five as multiples of the customer’s mean gap, and trailing counts from five minutes to thirty days divided by that customer’s longer-window rate, so a burst is measured against personal habit rather than a platform average.

Familiarity records, for eight entity types, whether the customer has used this value before, how deep the history runs and how long ago it was last seen, separating a dormant pairing that wakes from one in continuous use. *Relationship structure* counts first-time pairings inside a trailing window accounts newly acquired by a device over seven days, customers newly seen by a merchant within the hour so the counts do not inflate as the platform grows. *Concentration* is a Herfindahl index over prior device, merchant and location usage, since an unfamiliar device says far more against a concentrated profile than a scattered one, and *Movement* covers location changes per elapsed hour and device switching speed. Any quantity growing with calendar time is capped or ratioed, since lifetime counts leave their fitted range across a two-month forecast.

C. Fraud-Rate Encodings and Staleness Mitigation

Customer and device identifiers are excluded from the model, so their accumulated risk reaches it only through target encoding. We compute two views: an expanding smoothed fraud rate over labelled rows strictly earlier than the row being encoded, and a decayed rate weighting the same evidence exponentially with a 45-day time

constant, so a changed profile registers quickly. Each carries a support column, separating a rate backed by six observations from the same rate backed by six hundred. Smoothing shrinks toward the base rate [4], lighter on the two high-cardinality keys.

These encodings freeze on 15 July and are, by the close of scoring, up to 62 days out of date a condition no fold in either earlier pipeline had been scored under, so its cost was unknown. We measured it by fitting one model twice on identical rows, differing only in how the validation block’s encodings were built: *fresh*, up to each row’s own timestamp, as every walk-forward run reports it, and *frozen* at the fitting boundary, as a test row sees it. The difference was -0.0183 AP wider than the margin separating several leaderboard positions. We therefore added a member trained on the same matrix with every encoding deleted: it scores lower while encodings are fresh, but cannot lose anything as they age.

D. Rules, Validation, and Members

A scan of the highest-gain features found single thresholds isolating fraud at 60 to 70 times the base rate. A boosted tree locates such a cut only to its histogram resolution, so we supply it directly. Twelve rules cleared precision 0.25 and recall 0.005, firing on 1.58% of labelled rows and covering 39.9% of fraud, with thresholds fitted before the earliest fold opens.

Validation uses two expanding-window folds with embargo gaps and no shuffling. **Horizon** fits through 13 May and scores the following 62 days (240,065 rows), matching the forecast *distance* and carrying three times the positives. **Recent** fits through 25 May behind a 30-day embargo and scores 24 June to 15 July (82,762 rows), matching recent behavioural *content*. Neither is correct alone, so selection used $0.35 \text{ AP}_{\text{hor}} + 0.65 \text{ AP}_{\text{rec}}$, leaning on the fold that resembles the test period on both axes.

Four learners see those folds (Fig. 1): LightGBM on the full matrix [5]; the same configuration with all encodings removed; CatBoost at depth six; and a 384–192–96 multilayer perceptron with learned embeddings, trained on CPU with AdamW. Predictions are rank-transformed before combination, making the blend invariant to calibration, and weights come from coordinate ascent on the selection score, converging to 0.05, 0.20, 0.34 and 0.42. The network earns its place by disagreeing with axis-aligned trees in ways the blend exploits.

III. RESULTS AND DISCUSSION

Table I traces the three pipelines we built. The first used 175 features and four rolling 21-day folds, averaging 0.7678 AP over a wide spread (± 0.087): three folds scored above 0.81 while the fourth fell to 0.62, the first indication that the data changed character in late June. The second widened the feature set and added a 62-day horizon fold whose 0.7505 was a far more honest reading. The third dropped a redundant fold, added the per-entity encodings, the rules and two new members, lifting the horizon fold to 0.7568.

Local scores near 0.75 and a leaderboard score near 0.56 are not the same quantity: the folds score training-period blocks against encodings a few weeks old, while the leaderboard scores two months of drifted traffic against encodings frozen in mid-July. Our probe accounts for roughly 0.018 of that gap; the remainder is behaviour drift.

The more uncomfortable observation is how little the modelling work bought. From v2 to v3 the horizon fold improved by 0.0063 AP, about a third of it from the mined rules rather than any model change, and of twelve variants we tried recency weighting, re-ranking, drift pruning, segment-aware fold weights, exhaustive blend grids

TABLE I
PROGRESSION ACROSS THE THREE SYSTEM VERSIONS (LOCAL WALK-FORWARD PR-AUC)

	v1	v2	Proposed
Features	175	217	250 (+14 rules)
Members	LGB, XGB, Cat	LGB, XGB, Cat	LGB, Cat, MLP, no-enc.
Rolling-fold mean AP	0.7678	—	—
Horizon fold AP	—	0.7505	0.7568
Recent fold AP	—	0.6197	0.6181
Selection score	—	0.7251	0.6666
Staleness measured	no	no	yes (-0.0183)
Runtime (4 CPU cores)	35 min	55 min	73 min

TABLE II
FINAL LEADERBOARD, TOP 7 OF 36. LAST OF 36 SCORED 0.22093.

Rank	Team	Public	Private
1st	Error404	0.58235	0.58014
2nd	Overfit & Overcaffeinated	0.56548	0.56462
3rd	AL_Masaar	0.56217	0.56122
4th	COiN Lab (ours)	0.56217	0.56084
5th	Eldians	0.56189	0.55775
6th	BlackFlag	0.55838	0.55659
7th	DataX	0.55719	0.55658

almost none moved the score outside noise. Recency weighting was actively harmful, 0.7494 undecayed against 0.7481 at a 60-day half-life. The negative results were the more informative ones: once the causal feature layer was in place it did nearly all the work, and the estimator above it was close to interchangeable.

Table II gives the final standings. We placed 4th of 36 on both boards, the private score only 0.00133 below the public one, which suggests the time-aware validation did its job. The pipeline ran in 4,377 seconds on four CPU cores three minutes for features, twenty-four for the staleness probe, thirty for the members, the rest for refits and scoring with PyTorch reporting `cpu` throughout. A transformer might have recovered some of what these features encode, but at GPU hours per retrain and a far larger engineering surface. Against the winner that trade cost about 0.02 AP; what it bought is a system retrainable nightly on hardware a bank already owns, whose twelve rules read as auditable checks. Two limitations remain: selection leans 65% on a fold with about 1,330 positives, and we measured staleness without repairing it.

IV. CONCLUSION

We built a temporal fraud scorer from 250 strictly causal features, twelve mined rules and a four-member rank blend, running end to end on CPU in about 73 minutes and finishing 4th of 36 teams. Once the feature layer was in place, twelve attempts to improve the model moved the score less than one measurement of how much the encodings decay across the forecast gap. Where the signal must be reconstructed rather than consumed, building it correctly beats tuning the estimator, at a fraction of the compute.

REFERENCES

- [1] J. Davis and M. Goadrich, “The relationship between Precision-Recall and ROC curves,” *Proc. ICML*, 2006, pp. 233–240.
- [2] C. Bergmeir and J. M. Benítez, “On cross-validation for time series evaluation,” *Inf. Sci.*, vol. 191, pp. 192–213, 2012.
- [3] L. Grinsztajn *et al.*, “Why do tree-based models still outperform deep learning on tabular data?,” *Proc. NeurIPS*, 2022.
- [4] D. Micci-Barreca, “A preprocessing scheme for high-cardinality categorical attributes,” *ACM SIGKDD Explor.*, vol. 3, no. 1, pp. 27–32, 2001.
- [5] G. Ke *et al.*, “LightGBM: A highly efficient gradient boosting decision tree,” *Proc. NIPS*, 2017, pp. 3146–3154.